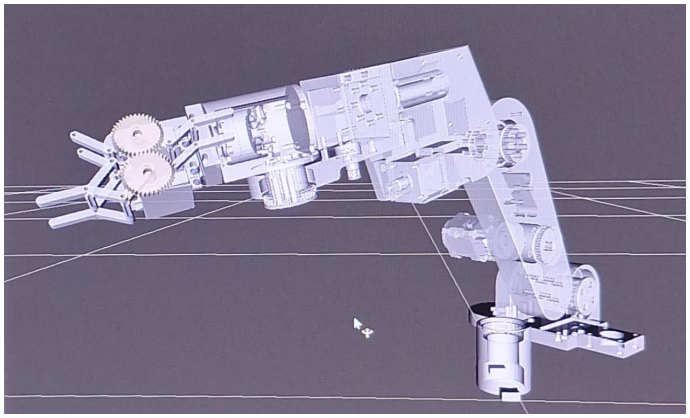
Entwicklung eines 6dof Roboterarms



ASIT-Projekt verfasst von

Matteo Gassler
Emanuel Käfer
Alexander Dick

Betreuer: Prof. Dr. Gerhard Gaube

HTBLA Saalfelden

Abteilung für Elektrotechnik

Abgabevermerk:

Saalfelden, 2 April 2026 übernommen von:

Vorwort

Vorwort Einfügen

--

Ort Datum Einfügen

Matteo Gassler & Emanuel Käfer & Alexander Dick

Aufgabenstellung

Da die Entwicklung unseres 6dof Roboterarms im Rahmen des Faches ASIT erfolgt und der Lernerfolg im Vordergrund steht konnten wir uns die Aufgabenstellung selbst stellen und hatten dabei keine großen Einschränkungen. Wir haben uns dazu entschieden das der Roboterarm als Mindestanforderung eine Pick and Place Operation, also ein Objekt aufheben und wo anders platzieren können soll. Als optionale weitere Herausforderung soll man über ein Graphical user Interface Koordinaten eingeben können zu denen sich der Arm dann bewegt.

--

Ort Datum Einfügen
Matteo Gassler & Emanuel Käfer & Alexander Dick

Inhaltsverzeichnis

Vorwort ii

Aufgabenstellung iii

1 Einleitung 1

2 Low Level Architektur und Steuerung 2

2.1 Implementierung der 6 Achsen und Skalierbarkeit 2

2.2 Initialisierung und SPI-Kommunikation der Motortreiber 3

2.3 Finite State Machine (Endlicher Zustandsautomat) 5

2.4 Kinematische Referenzierungsfahrt (Homing) 7

2.5 Verbindung mir Raspberry Pi und normaler Betrieb 8

3 High Level Software 9

4 Hardware 10

5 Elektrisches 11

Quellenverzeichnis 13

1 Einleitung

2 Low Level Architektur und Steuerung

Dieses Kapitel beschreibt die Implementierung der zentralen Low-Level-Robotersteuerung, die auf einem Mikrocontroller läuft. Sie stellt das funktionale Bindeglied zwischen der mechanischen Konstruktion (siehe 4) und der elektrischen Leistungselektronik (siehe 5) dar. Die Firmware wurde in Bare-Metal-C für einen STM32F446-Mikrocontroller in der Entwicklungsumgebung STM32CubeIDE entwickelt. Ziel der Softwarearchitektur ist es, harte Echtzeitanforderungen für die synchrone Ansteuerung von sechs Schrittmotoren zu erfüllen und gleichzeitig eine hochgradig skalierbare Basis für zukünftige Pfadplanungen zu schaffen.

2.1 Implementierung der 6 Achsen und Skalierbarkeit

Bei der Ansteuerung eines 6-Achs-Roboterarms führt die Zuweisung von Mikrocontroller-Pins als Einzelvariablen und die statische Zuweisung von Peripherie-Registern schnell zu eingeschränktem und fehleranfälligem Programmcode. Um dies zu vermeiden, wurde eine Array-basierte Datenstruktur implementiert. Anstatt jede Achse separat zu adressieren, werden Pointer auf die entsprechenden Hardware-Timer (`TIM_HandleTypeDef`), PWM-Kanäle sowie die GPIO-Ports für Richtungs- (`DIR`) und Chip-Select-Signale (`CS`) in globalen Arrays gekapselt.

Durch diese Entkopplung von physikalischer Hardware und Steuerungslogik kann jede Achse über einen simplen Index angesprochen werden. Dies ermöglicht es, Kernfunktionen wie das Starten und Stoppen der Motoren oder das Homen über alle Treiber mittels einfacher Schleifenkonstrukte umzusetzen, was die Skalierbarkeit und Wartbarkeit des Systems massiv erhöht.

2.2 Initialisierung und SPI-Kommunikation der Motortreiber

2.2.1 SPI Protokoll

Das Seriel Peripheral Interface (SPI) ist ein Serielles Hochgeschwindigkeit Bussystem das vorwiegend zur Kommunikation von Mikrocontrollern mit Peripheriegeräten auf Platinen dient. SPI ist full duplex fähig, das heißt das bei jedem Takt 1 Bit vom Master an den Slave und zurück gesendet wird. Es besteht aus 3 Leitungen MOSI (Master Out $\longrightarrow$ Slave In), MISO (Master In $\longleftarrow$ Slave Out) und SCK (Serial Clock). Dazu kommt eine CS (Chip Select) Leitung die vom Master zu jeden Slave einzeln gehen muss. Nur wenn diese CS Leitung LOW ist sendet/empfängt der Slave Daten. Die Kommunikation erfolgt meist so das 2 Übertragungen nötig sind. In der ersten Übertragung sendet der Master ein Kommando an den Slave danach, da der Slave aber noch nicht bereit für eine Sinnvolle Rückmeldung ist sendet er nur Datenmüll. In der 2 Übertragung sendet der Master einen beliebigen Datensatz an den Slave, dieser hat inzwischen zeit gehabt seine Antwort auf das Kommando des Masters bereitzustellen und sendet diese zum Master. Die Übertragungen sind jeweils synchron, heißt der Master und der Slave senden/empfangen synchron Daten.

SPI-Mode

Es gibt 4 SPI Modis Mode 0-3. Diese Beschreiben sich wiefolgt:
CPOL (Clock Polarity):

- 0: Takt ist in Ruhe LOW, ein Wechsel auf HIGH zählt als steigende Taktflanke
- 1: Takt ist invertiert: in Ruhe HIGH, ein Wechsel auf LOW zählt als steigende Taktflanke

CPHA (Clock Phase):

- 0: Daten werden bei steigender Taktflanke (=abh. von CPOL) eingelesen, bei fallender ausgegeben
- 1: Daten werden bei fallender Taktflanke eingelesen, bei steigender ausgegeben

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Abbildung 2.1: Diagramm der SPI Modi

2.2.2 Initialisierung der Motortreiber

Die verwendeten Schrittmotortreiber vom Typ TMC5160 in Kombination mit NEMA-17 Schrittmotoren erfordern eine initiale Konfiguration ihrer internen Register, um Parameter wie den Haltestrom, Run-Current und den Chopper-Modus festzulegen. Diese Kommunikation erfolgt über einen geteilten SPI Bus. Um die Treiber individuell adressieren zu können, wird vor jedem Datentransfer der spezifische Chip-Select (`CS`) Pin der Zielachse auf ein niedriges Potential (LOW) gezogen. Die TMC5160 erfordern den Betrieb in SPI Mode 3 (siehe 2.1)

Um Fehler direkt beim Initialisieren der Treiber zu erkennen, wurde eine Validierung eingebaut, ohne die der nächste Schritt nicht erreicht werden kann. Nach dem Schreiben der Konfigurationsdaten liest der Mikrocontroller das globale Statusregister (`GSTAT`) sowie aktuelle Treiberzustände zurück. Nur wenn das System valide Antworten (`GSTAT` = 0) detektiert, wird die Steuerung freigegeben. Störsignale oder fehlende Hardware-Verbindungen werden so unmittelbar beim Systemstart erkannt und führen zu einem sicheren Systemabbruch.

Die gewählte Konfiguration der Motortreiber ist wie folgt:

Tabelle 2.1: Konfiguration der TMC5160T Register via SPI

Register (Wert)	Funktion
GSTAT (0x01) = 0x00000007	Setzt das globale Statusregister zurück. Löscht Fehlerspeicher für Reset, Unterspannung der Charge-Pump und Driver-Shutdown vor dem Testlauf.
GCONF (0x00) = 0x00000004	Aktiviert das Bit pdn_disable. Schaltet den Treiber in den internen Sensing-Modus um und erzwingt die Kommunikation rein über die SPI-Schnittstelle.
IHOLD_IRUN (0x10) = 0x00061008	Konfiguriert den Motorstrom: Setzt den Fahrstrom (IRUN) auf ca. 50 %, den Haltestrom im Stillstand (IHOLD) auf ca. 25 % sowie eine Verzögerung (IHOLDDELAY) für die Stromabsenkung.
CHOPCONF (0x6C) = 0x14010053	Konfiguration des Chopper-Verfahrens. Aktiviert den kraftvollen *SpreadCycle*-Modus, stellt die Schrittauflösung auf 16 Mikroschritte (mit automatischer Interpolation auf 256 Schritte) und definiert das Signal-Timing (*TOFF*, *TBL*).
PWMCONF (0x70) = 0xC10D0024	Hinterlegt die Parameter für den flüsterleisen *StealthChop2*-Modus (Spannungsamplitude, Frequenz, Gradienten) für einen potenziellen, geschwindigkeitsabhängigen Moduswechsel.

2.3 Finite State Machine (Endlicher Zustandsautomat)

Um die auf dem Mikrocontroller laufende Software übersichtlich zu gestalten und ein Selbstblockieren der Software zu verhindern, kommt eine Finite State Machine (FSM) zum Einsatz. Eine FSM ist ein mathematisches Modell für Systeme, die sich zu jedem Zeitpunkt in genau einem von mehreren Zuständen befinden. Eine FSM besteht immer aus den folgenden Komponenten:

- Zustände (States): Die verschiedenen Phasen des Systems.
- Startzustand (Initial State): Der Zustand, in dem die FSM beginnt.
- Übergänge (Transitions): Die Bedingungen für den Wechsel von Zuständen.
- Ereignisse (Inputs): Die Ereignisse, die einen Wechsel bewirken.
- Aktionen (Outputs): Die Ausgaben der FSM.

2.3.1 Zustände

Für unseren Roboterarm wurden die Zustände STATE_INIT, STATE_HOMING, STATE_HOMING_OFFSET, STATE_READY und STATE_ERROR gewählt. Als Startzustand ist STATE_INIT definiert, da vor jedem Betrieb die Treiber initialisiert werden müssen. Die Funktionen der Zustände gestalten sich wie folgt:

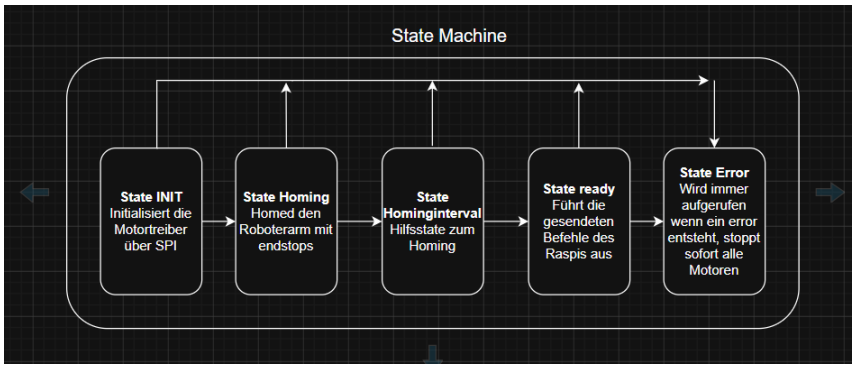


Abbildung 2.2: Diagramm der Finite State Machine

Tabelle 2.2: Funktionen der Zustände der Finite State Machine

Zustand	Funktion
`STATE_INIT`	Initialisiert die Motortreiber über SPI, um die nötigen Parameter zu konfigurieren.
`STATE_HOMING`	Hier passiert der erste Teil der kinematischen Referenzfahrt, die den Roboter in die Home-Position bringt. Er fährt dabei den Endschalter der jeweiligen Achse an.
`STATE_HOMING_OFFSET`	Hier passiert der zweite Teil der kinematischen Referenzfahrt. Der Roboter fährt vom Endschalter in die vordefinierte Home-Position.
`STATE_READY`	Hier erfolgt der eigentliche Betrieb des Roboters. Der Mikrocontroller empfängt Befehle und führt diese aus, um den Roboter wie gewünscht zu bewegen.
`STATE_ERROR`	In diesen Zustand gelangt man nur im Fehlerfall. Es werden alle Motoren gestoppt und eine Fehlermeldung ausgegeben.

2.4 Kinematische Referenzierungsfahrt (Homing)

Da Schrittmotoren über keine absolute Positionscodierung verfügen und aus Kostengründen auf absolute Encoder verzichtet wurde, ist der Systemzustand nach jedem Neustart undefiniert. Um den vordefinierten kinematischen Nullpunkt des Roboters zu ermitteln, muss zwingend eine Kalibrierungsfahrt (Homing) durchgeführt werden.

2.4.1 Interrupt-Verarbeitung und Hardware-Entprellung

Für die Erfassung der Endschalter kommen externe Interrupts (`EXTI`s) zum Einsatz. `EXTI`s sparen erheblichen Rechenaufwand, da der Prozessor nicht permanent den Pin abfragen muss (Polling). Sobald eine Flanke erkannt wird, wird ein Interrupt-Request ausgelöst, der das Hauptprogramm pausiert und eine spezifische Interrupt-Service-Routine (ISR) aufruft. Diese Funktion setzt ein Flag, um dem Hauptprogramm zu signalisieren, dass der Endschalter getroffen wurde, das zu fahrende Offset zu berechnen ist und die Fahrtrichtung umgekehrt werden muss.
Um das mehrfache Auslösen des Interrupts zu verhindern, das durch sogenanntes Bouncen bei mechanischen Schaltern auftreten kann, ist in der Interrupt-Funktion eine Software-Entprellung eingebettet, die nach einem registrierten Interrupt alle weiteren Signale für 30 ms ignoriert.
Der Code dafür sieht wie folgt aus:

```
1 static uint32_t last = 0;
2 uint32_t now = HAL_GetTick();
3 if (now - last < 30) return;
4 last = now;
```

Programmcode 2.1: Debouncing für Interrupt-Verarbeitung

2.4.2 Berechnung kinematischer Offsets

Der mechanische Nullpunkt, an dem sich die Endschalter befinden, entspricht meist nicht dem exakten kinematischen Nullpunkt des Roboters. Dies führt zu einem mathematischen Offset, den es auszugleichen gilt. Um diesen auszugleichen, muss die Winkeldifferenz in die entsprechende Schrittanzahl für die Motoren umgerechnet werden. Die Formel hierfür lautet:

$$\text{steps to move} = \frac{\Delta\text{rad}}{2 \cdot \pi} \cdot 200 \cdot 16 \cdot 20 \tag{2.1}$$

- Δrad: Die Differenz des Offsets und der Home-Position in Radiant.
- 200: Die Vollschritte, die ein Schrittmotor pro Umdrehung besitzt.
- 16: Die eingestellte Mikroschritt-Auflösung pro Vollschritt.
- 20: Die mechanische Getriebeübersetzung (1:20) bei allen Achsen.

2.4.3 Sequenzieller Homing-Ablauf

Die Referenzierung des 6-Achs-Systems erfolgt sequenziell, gesteuert durch eine globale Index-Variable (`current_homingaxis`), beginnend bei der Basis (Achse 0) bis zum Endeffektor (Achse 5). Der Ablauf für eine einzelne Achse gliedert sich dabei in folgende Phasen:
Zunächst verfährt die Achse im Zustand `STATE_HOMING` mit konstanter Geschwindigkeit auf den Endschalter zu. Nach Auslösen des Interrupts berechnet das System den erforderlichen Offset, invertiert das Richtungssignal (`DIR`) und wechselt in den Zustand `STATE_HOMING_OFFSET`. Erst wenn die exakt berechnete Schrittzahl für den Offset über den Timer-Interrupt abgefahren wurde, gilt die Achse als referenziert. Der Index wird inkrementiert und die Prozedur für das nächste Gelenk gestartet, bis das System vollständig kalibriert ist und in den Betriebszustand `STATE_READY` übergeht.

2.5 Verbindung mir Raspberry Pi und normaler Betrieb

Nach dem ausführen der Kinematischen Referenzfahrt und dem erfolgreichen erreichen der Home Position jeder Achse erreicht der Roboterarm den Zustand `STATE_READY`. Das ist der Zustand in dem der Normale Betrieb stattfindet. Der STM bekommt über UART Befehle von einem Raspberry PI der die High Level Software wie z.B Berechnung der Kinematik übernimmt. Der Raspberry PI sendet die geforderte Position des Arm als Array der 6 zu erreichenden Joint Winkel. Der STM errechnet dann aus diesen Daten die Schritte und Richtung die Jeder Motor fahren muss. Sobald die gewünschte Position erreicht ist sendet der STM eine Rückmeldung das die Position erreicht ist. So werden Aufträge mit mehreren zu erreichenden Wegpunkten sequentiell abgearbeitet.

3 High Level Software

4 Hardware

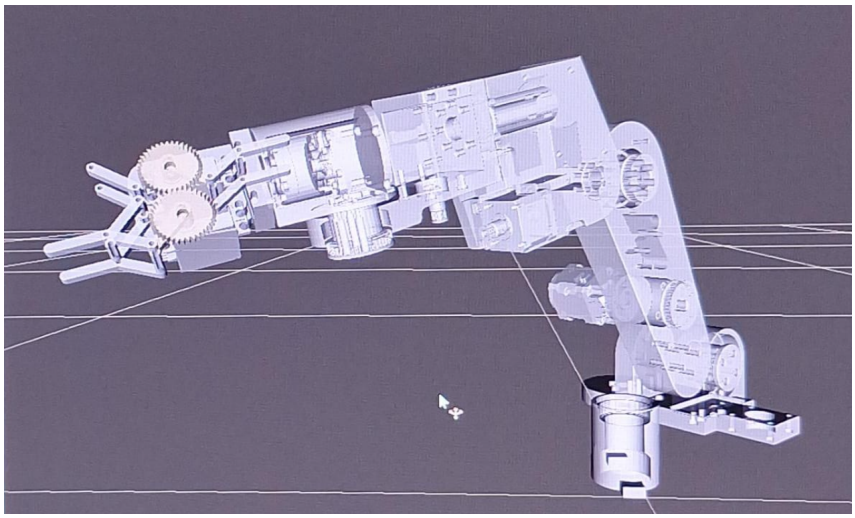


Abbildung 4.1: Blockdiagramm des 6-DOF-Roboterarms

5 Elektrisches

Die Entwicklung moderner Roboterarme basiert stark auf den Prinzipien der kollaborativen Robotik,[1] bei denen Sicherheit und Flexibilität im Vordergrund stehen [2].

Abbildungsverzeichnis

2.1 Diagramm der SPI Modi 4
2.2 Diagramm der Finite State Machine 6

4.1 Blockdiagramm des 6-DOF-Roboterarms 10

Quellenverzeichnis

[1] *WS2812B Intelligent control LED integrated light source Features and Benefits*. Adresse: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf

[2] *WS2812 Ansteuerung – Mikrocontroller.net*, Mikrocontroller.net, 2025. besucht am 10. Feb. 2025. Adresse: https://www.mikrocontroller.net/articles/WS2812_Ansteuerung?